SISTEMA DE GESTIÓN DE BIBLIOTECA VIRTUAL

FASE 4: DISEÑO DE BASE DE DATOS RELACIONAL Y PERSISTENCIA SQL

Arquitectura de Datos en Tercera Forma Normal (3FN), Modelos Entidad-Relación y Script DDL/DML en MySQL InnoDB

INTEGRANTES DEL EQUIPO (GRUPO 08):

JUAN DAVID ROMERO CALDERÓN

JUAN DIEGO OTECA PEDREROS

JHOJAN JAVIER MELO AVELLA

INSTRUCTOR TÉCNICO:

ING. CARLOS JULIO CADENA

SERVICIO NACIONAL DE APRENDIZAJE (SENA)
CENTRO DE LA INDUSTRIA, LA EMPRESA Y LOS SERVICIOS (CIES)
REGIONAL HUILA
TECNOLOGÍA EN ANÁLISIS Y DESARROLLO DE SOFTWARE (ADSO)
FICHA DE CARACTERIZACIÓN: 3239188
NEIVA, HUILA — 2026

TABLA DE CONTENIDO

1. INTRODUCCIÓN Y JUSTIFICACIÓN DEL MODELO 3

2. NIVELES DE MODELADO DE DATOS 4

2.1. Modelo Conceptual (Entidades y Relaciones de Negocio) 4

2.2. Modelo Lógico Relacional (Normalizado e Independiente del Motor) 4

2.3. Tabla de Equivalencias de Dominio (Mapeo Español - Inglés) 5

3. PROCESO DE NORMALIZACIÓN (1FN, 2FN Y 3FN) 6

3.1. Primera Forma Normal (1FN) 6

3.2. Segunda Forma Normal (2FN) 6

3.3. Tercera Forma Normal (3FN) 6

4. MODELO FÍSICO RELACIONAL Y DIAGRAMA ERD (MYSQL / MARIADB) 7

5. DICCIONARIO DE DATOS DEL SISTEMA 8

5.1. Entidad: categories (Categorías Temáticas) 8

5.2. Entidad: authors (Autores Registrados) 8

5.3. Entidad: books (Catálogo de Ejemplares Físicos) 9

5.4. Entidad: users (Usuarios Prestatarios) 10

5.5. Entidad: loans (Registro Transaccional de Préstamos) 10

6. IMPLEMENTACIÓN DE REGLAS DE NEGOCIO Y RESTRICCIONES EN SQL 11

6.1. Matriz de Trazabilidad de Reglas de Negocio a Constraints SQL 11

6.2. Justificación Arquitectónica del Estado OVERDUE y Columna Virtual 11

7. SCRIPT DDL/DML E INSTRUCCIONES DE DESPLIEGUE (MYSQL Y DOCKER) 12

7.1. Extracto de Definición Estructural DDL (database/schema.sql) 12

7.2. Procedimiento de Despliegue Local y en Contenedor Docker 12

8. CONCLUSIONES DE LA FASE 13

ÍNDICE DE FIGURAS Y TABLAS

Figura 1. Modelo Conceptual Entidad-Relación del dominio bibliotecario **4**

Figura 2. Modelo Lógico Relacional normalizado en Tercera Forma Normal (3FN) **5**

Figura 3. Modelo Físico Relacional con tipos de datos MySQL InnoDB e índices **7**

Tabla 1. Matriz de equivalencia canónica POO (Español) y SQL (Inglés) **5**

Tablas 2 a 6. Diccionario de datos (categories, authors, books, users, loans) **8–10**

Tabla 7. Trazabilidad entre reglas de negocio (RN-01 a RN-05) y restricciones SQL **11**

1. INTRODUCCIÓN Y JUSTIFICACIÓN DEL MODELO

La Fase 4 del proyecto formativo establece la arquitectura de datos y la capa de persistencia relacional del **Sistema de Biblioteca Digital**, proporcionando un modelo de almacenamiento estructurado, normalizado y optimizado sobre el motor transaccional **MySQL / MariaDB (InnoDB)**.

Mientras que las Fases 1, 2 y 3 definieron el alcance funcional mediante historias de usuario (HU-01 a HU-13), la especificación de requisitos bajo el estándar IEEE 830 y el modelado orientado a objetos en lenguaje UML, la presente fase materializa la transición desde una estructura volátil en memoria hacia un esquema de base de datos relacional persistente que garantiza las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad).

En estricto cumplimiento con los lineamientos técnicos del programa de Tecnología en Análisis y Desarrollo de Software (ADSO) del SENA y las directrices de arquitectura del instructor técnico, el diseño de persistencia se fundamenta en cinco pilares de ingeniería:

1. **Modelado Arquitectónico en Tres Capas de Abstracción:** El diseño se desglosa de manera explícita e incremental en el *Modelo Conceptual* (orientado a las entidades del negocio), el *Modelo Lógico Relacional* (orientado a la estructura normalizada independiente del SGBD) y el *Modelo Físico Relacional* (específico para el motor InnoDB de MySQL 8.0).
2. **Estándar Internacional de Nomenclatura en Base de Datos:** Siguiendo las buenas prácticas de la industria de software, los identificadores de las tablas se declaran en **plural e inglés** (`categories`, `authors`, `books`, `users`, `loans`), mientras que los nombres de las columnas se definen en **singular, inglés y formato *snake_case*** (por ejemplo, `author_code`, `publication_year`, `document_number`), documentando formalmente su trazabilidad con las clases en español diseñadas en la Fase 3.
3. **Rigor en Normalización hasta Tercera Forma Normal (3FN):** La estructura elimina por completo los grupos repetitivos, las dependencias funcionales parciales y las dependencias transitivas, segregando los datos maestros de las categorías temáticas y los autores respecto al inventario de libros y al registro histórico de préstamos.
4. **Integridad Referencial Estricta y Patrón de Baja Lógica (*Soft Delete*):** Todas las relaciones de clave foránea aplican la política `ON UPDATE CASCADE` y `ON DELETE RESTRICT`. Para impedir la pérdida de trazabilidad histórica cuando un ejemplar bibliográfico sufre deterioro o extravío (HU-08 CA-2 y RN-05), el sistema implementa el atributo booleano `is_deleted`, ocultando el libro del catálogo operativo sin destruir los registros de préstamos asociados en el pasado.
5. **Correspondencia de Inventario y Unicidad Física de Préstamo Activo:** Conforme a la restricción de alcance del Producto Mínimo Viable (RS-04), cada tupla en la tabla `books` representa un ejemplar físico único. Asimismo, se incorpora una columna virtual generada (`active_book_id`) con restricción `UNIQUE` en la tabla `loans`, garantizando a nivel de motor de base de datos que ningún libro pueda mantener dos préstamos activos de manera simultánea.

2. NIVELES DE MODELADO DE DATOS

Siguiendo la metodología de ingeniería de bases de datos relacionales, el diseño evoluciona desde la comprensión semántica del dominio hasta su especificación relacional normalizada.

2.1. Modelo Conceptual (Entidades y Relaciones de Negocio)

El modelo conceptual representa las entidades fundamentales del dominio bibliotecario y sus asociaciones semánticas sin acoplarse a tipos de datos físicos ni claves foráneas:

- **CATEGORIA:** Clasificación temática que agrupa las obras del acervo bibliográfico.
- **AUTOR:** Creador intelectual de una o varias obras, identificado mediante un código estándar internacional.
- **LIBRO:** Ejemplar bibliográfico físico registrado en el inventario de la biblioteca.
- **USUARIO:** Persona natural afiliada y habilitada para solicitar servicios de préstamo.
- **PRESTAMO:** Evento transaccional que vincula temporalmente un ejemplar con un usuario prestatario bajo un plazo determinado.

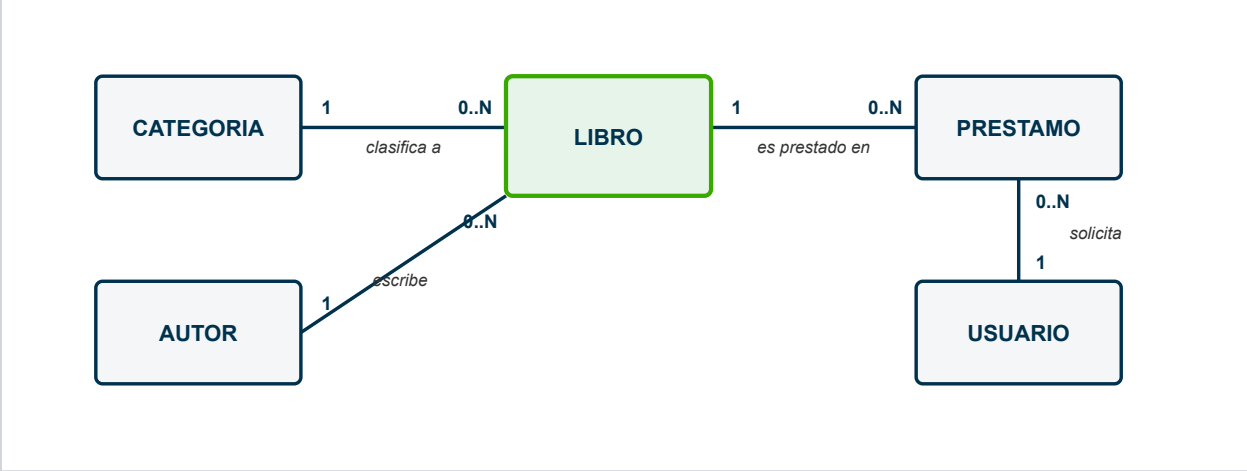


Figura 1. Modelo Conceptual Entidad-Relación del dominio bibliotecario.
Fuente: Elaboración propia del Equipo 08 mediante Mermaid.

2.2. Modelo Lógico Relacional (Normalizado e Independiente del Motor)

El modelo lógico estructura los atributos de cada entidad en Tercera Forma Normal (3FN), definiendo claves primarias (PK), claves candidatas únicas (UK) y claves foráneas (FK) que materializan las relaciones de cardinalidad uno a muchos (1:N):

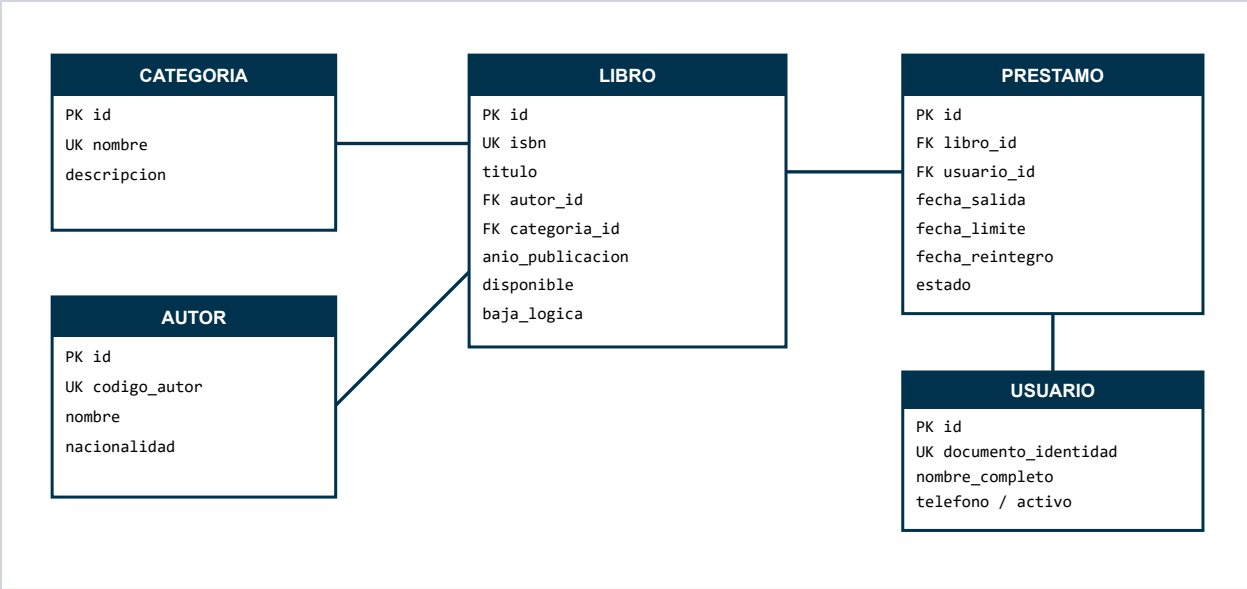


Figura 2. Modelo Lógico Relacional normalizado en Tercera Forma Normal (3FN).
Fuente: Elaboración propia del Equipo 08 mediante Mermaid.

2.3. Tabla de Equivalencias de Dominio (Mapeo Español - Inglés)

Con el propósito de cumplir la convención de persistencia en inglés (tablas en plural y columnas en singular) manteniendo trazabilidad directa con los artefactos en español de las Fases 1, 2 y 3, se define la matriz canónica de correspondencia:

Tabla 1. Matriz de equivalencia canónica entre el modelo de objetos POO (Español) y el modelo relacional SQL (Inglés).

Clase POO (Fase 3 - ES)	Tabla SQL (Fase 4 - EN)	Mapeo de Atributos Principales	Justificación de Ingeniería
Categoria	`categories`	`nombre` → `name` `descripcion` → `description`	Catálogo maestro normalizado en plural con atributos descriptivos en inglés.

Autor	`authors`	`identificador` → `author_code`
		`nombre` → `name`
		`nacionalidad` → `nationality`
			Soporte unificado para códigos internacionales normalizados (ISNI u ORCID).

Libro	`books`	`titulo` → `title`
		`isbn` → `isbn`
		`anioPublicacion` → `publication_year`
		`disponible` → `is_available`
		Baja Lógica → `is_deleted`
			Representación de ejemplares individuales (RS-04) con bandera de baja lógica para preservar auditoría.

Clase POO (Fase 3 - ES)	Tabla SQL (Fase 4 - EN)	Mapeo de Atributos Principales	Justificación de Ingeniería

Usuario	`users`	`documentoIdentidad` → `document_number`
		`nombre` → `full_name`
		`telefono` → `phone`
		`activo` → `is_active`
			Padrón de usuarios prestatarios validados por documento de identidad único.

Prestamo	`loans`	`fechaPrestamo` → `loan_date`
		`fechaLimiteDevolucion` → `due_date`
		`fechaDevolucion` → `return_date`
		`estado` → `status`
			Registro transaccional histórico con columna virtual generada para control de unicidad activa.

3. PROCESO DE NORMALIZACIÓN (1FN, 2FN Y 3FN)

El esquema relacional fue sometido a un proceso sistemático de refinamiento aplicando las tres primeras formas normales de Codd, con el objetivo de erradicar anomalías de inserción, actualización y borrado.

3.1. Primera Forma Normal (1FN)

- **Criterio Teórico:** Una relación se encuentra en Primera Forma Normal (1FN) si y solo si todos sus atributos contienen valores estrictamente atómicos (indivisibles), no existen grupos repetitivos ni arreglos multivaluados en una misma celda, y cada tupla es identificable mediante una clave primaria única.
- **Aplicación Práctica en el Esquema:**
  - En la tabla `authors`, el código normalizado (`author_code`), el nombre completo (`name`) y la nacionalidad (`nationality`) se descomponen en columnas atómicas independientes.
  - En la tabla `books`, el código internacional `isbn` se almacena como un valor escalar único por ejemplar, acompañado de una clave primaria subrogada autoincremental (`id INT AUTO_INCREMENT`).
  - Se descarta por completo el almacenamiento de colecciones de libros prestados dentro del registro del usuario; cada evento de préstamo se independiza en una fila atómica dentro de la entidad transaccional `loans`.

3.2. Segunda Forma Normal (2FN)

- **Criterio Teórico:** Una relación se encuentra en Segunda Forma Normal (2FN) cuando, además de satisfacer 1FN, todo atributo que no forma parte de la clave primaria depende funcionalmente de forma completa de la clave primaria entera, y no de una fracción de ella (ausencia de dependencias funcionales parciales).
- **Aplicación Práctica en el Esquema:**
  - Las cinco tablas del sistema implementan claves primarias simples de una sola columna (`id INT AUTO_INCREMENT`), lo que garantiza estructuralmente la inexistencia de dependencias parciales sobre claves compuestas.
  - Los atributos descriptivos de clasificación temática (`name` y `description`) dependen únicamente del identificador de la categoría y no del libro, razón por la cual se aíslan en la tabla maestra `categories` y se enlazan desde `books` mediante la clave foránea `category_id`.

3.3. Tercera Forma Normal (3FN)

- **Criterio Teórico:** Una relación se encuentra en Tercera Forma Normal (3FN) cuando está en 2FN y ningún atributo no clave depende transitivamente de la clave primaria a través de otro atributo no clave (es decir, para toda dependencia funcional no trivial $X \rightarrow Y$, X es superclave).
- **Aplicación Práctica en el Esquema:**
  - **Desacoplamiento en `books`:** Si el nombre del autor (`name`) o su nacionalidad (`nationality`) se almacenaran directamente dentro de la tabla `books`, se produciría la dependencia transitiva `books.id → author_code → (name, nationality)`. Al extraer dichos atributos hacia la entidad maestra `authors` y conservar únicamente la clave foránea `author_id` en `books`, se elimina toda redundancia y cualquier actualización biográfica del autor se propaga instantáneamente sin modificar el catálogo de libros.
  - **Desacoplamiento en `loans`:** La tabla transaccional de préstamos no duplica el nombre del usuario, su teléfono ni el título del libro. Todos estos datos maestros se consultan mediante operaciones `INNER JOIN` a través de las claves foráneas `user_id` y `book_id`, asegurando consistencia absoluta en el historial.

4. MODELO FÍSICO RELACIONAL Y DIAGRAMA ERD (MYSQL / MARIADB)

El modelo físico traduce el diseño lógico al dialecto nativo de **MySQL 8.0 / MariaDB** bajo el motor de almacenamiento transaccional **InnoDB** y codificación `utf8mb4_unicode_ci`. El diagrama incorpora las restricciones de nulidad, tipos de datos exactos, marcas de auditoría temporal (`created_at`) y la columna virtual generada para el control de préstamos activos:

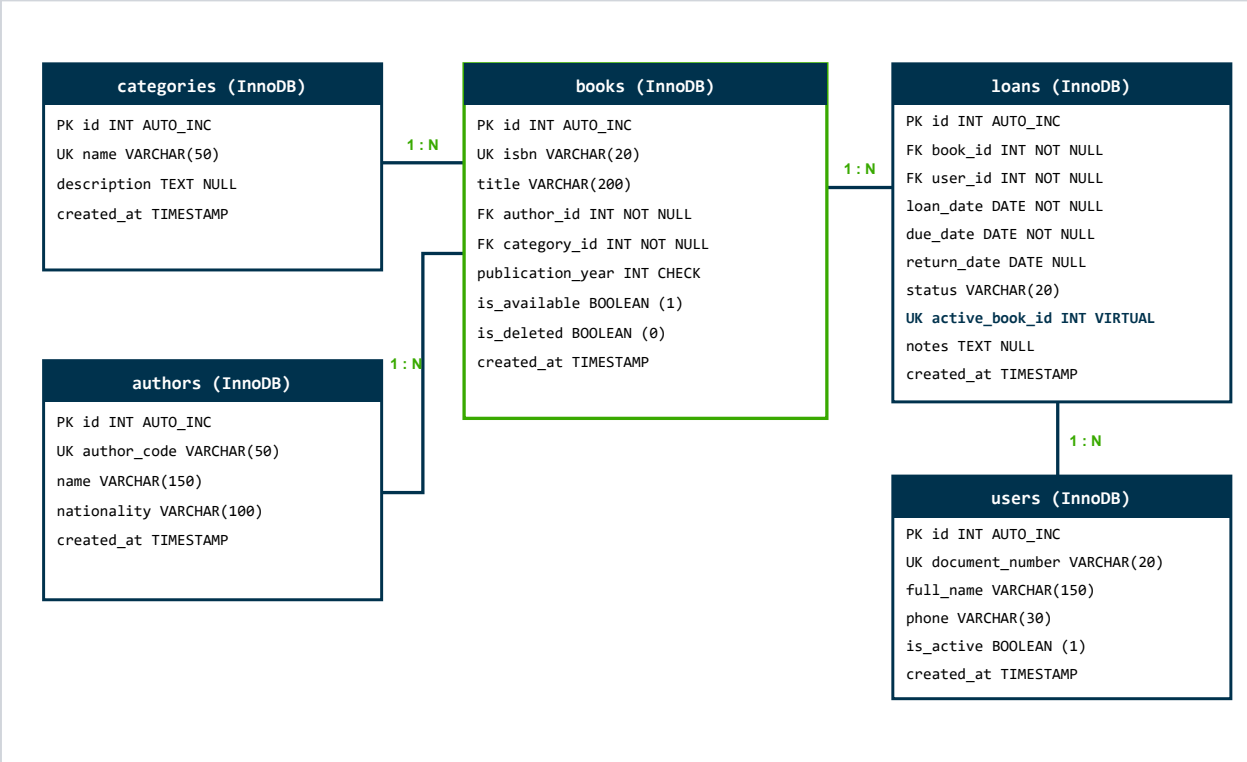


Figura 3. Modelo Físico Relacional con tipos de datos MySQL InnoDB e índices.
Fuente: Elaboración propia del Equipo 08 a partir de database/schema.sql.

Adicionalmente, el modelo físico contempla tres índices secundarios de rendimiento para acelerar las búsquedas frecuentes exigidas por el requerimiento no funcional RNF-01:

- `idx_books_title` sobre `books(title)`: optimiza la búsqueda parcial o exacta de libros por título (HU-07).
- `idx_loans_status` sobre `loans(status)`: agiliza el filtrado de préstamos activos frente a préstamos históricos.
- `idx_loans_due_date` sobre `loans(due_date)`: acelera el ordenamiento cronológico por fecha de vencimiento (HU-09).

5. DICCIONARIO DE DATOS DEL SISTEMA

A continuación se detalla la estructura técnica de cada una de las cinco tablas relacionales que conforman la base de datos `digital_library`.

5.1. Entidad: categories (Categorías Temáticas)

Tabla 2. Diccionario de datos de la tabla categories.

Campo	Tipo de Dato	Nulo	Clave	Valor por Defecto	Descripción Técnica
`id`	`INT`	NO	PK	`AUTO_INCREMENT`	Identificador único autoincremental de la categoría temática.
`name`	`VARCHAR(50)`	NO	UK	—	Nombre único de clasificación bibliográfica (ej. Novela, Ingeniería).
`description`	`TEXT`	SÍ	—	`NULL`	Descripción del alcance temático de la categoría.
`created_at`	`TIMESTAMP`	NO	—	`CURRENT_TIMESTAMP`	Marca de tiempo de auditoría de creación del registro.

5.2. Entidad: authors (Autores Registrados)

Tabla 3. Diccionario de datos de la tabla authors.

Campo	Tipo de Dato	Nulo	Clave	Valor por Defecto	Descripción Técnica
`id`	`INT`	NO	PK	`AUTO_INCREMENT`	Identificador numérico interno del autor.
`author_code`	`VARCHAR(50)`	NO	UK	—	Código estándar internacional ISNI u ORCID único (RN-02).
`name`	`VARCHAR(150)`	NO	—	—	Nombre completo o seudónimo oficial del autor.
`nationality`	`VARCHAR(100)`	NO	—	—	País de origen o nacionalidad del autor.
`created_at`	`TIMESTAMP`	NO	—	`CURRENT_TIMESTAMP`	Fecha y hora de registro del autor en el catálogo.

5.3. Entidad: books (Catálogo de Ejemplares Físicos)

Tabla 4. Diccionario de datos de la tabla books.

Campo	Tipo de Dato	Nulo	Clave	Valor por Defecto	Descripción Técnica
`id`	`INT`	NO	PK	`AUTO_INCREMENT`	Identificador único de inventario del ejemplar físico.
`isbn`	`VARCHAR(20)`	NO	UK	—	Código Internacional Normalizado del Libro único (RN-01).
`title`	`VARCHAR(200)`	NO	—	—	Título completo de la obra bibliográfica.
`author_id`	`INT`	NO	FK	—	Clave foránea que referencia a `authors(id)`.
`category_id`	`INT`	NO	FK	—	Clave foránea que referencia a `categories(id)`.
`publication_year`	`INT`	NO	—	—	Año de edición validado en el rango 1000 a 2100.
`is_available`	`BOOLEAN`	NO	—	`TRUE`	Estado físico: `1` = Disponible en estantería, `0` = En préstamo.
`is_deleted`	`BOOLEAN`	NO	—	`FALSE`	Bandera de baja lógica: `0` = Activo, `1` = Dado de baja (HU-08).
`created_at`	`TIMESTAMP`	NO	—	`CURRENT_TIMESTAMP`	Marca temporal de alta en el catálogo.

5.4. Entidad: users (Usuarios Prestatarios)

Tabla 5. Diccionario de datos de la tabla users.

Campo	Tipo de Dato	Nulo	Clave	Valor por Defecto	Descripción Técnica
`id`	`INT`	NO	PK	`AUTO_INCREMENT`	Identificador numérico interno del usuario prestatario.
`document_number`	`VARCHAR(20)`	NO	UK	—	Número de cédula o documento de identidad único (RN-03).
`full_name`	`VARCHAR(150)`	NO	—	—	Nombres y apellidos completos del afiliado.
`phone`	`VARCHAR(30)`	NO	—	—	Número telefónico de contacto para notificaciones.
`is_active`	`BOOLEAN`	NO	—	`TRUE`	Estado de habilitación para préstamos (`1` = Activo, `0` = Inactivo - RN-04).
`created_at`	`TIMESTAMP`	NO	—	`CURRENT_TIMESTAMP`	Fecha y hora de afiliación a la biblioteca.

5.5. Entidad: loans (Registro Transaccional de Préstamos)

Tabla 6. Diccionario de datos de la tabla loans.

Campo	Tipo de Dato	Nulo	Clave	Valor por Defecto	Descripción Técnica
`id`	`INT`	NO	PK	`AUTO_INCREMENT`	Número consecutivo único de la transacción de préstamo.
`book_id`	`INT`	NO	FK	—	Referencia al ejemplar prestado en `books(id)`.
`user_id`	`INT`	NO	FK	—	Referencia al usuario responsable en `users(id)`.
`loan_date`	`DATE`	NO	—	—	Fecha efectiva de salida del ejemplar bibliográfico.

Campo	Tipo de Dato	Nulo	Clave	Valor por Defecto	Descripción Técnica
`due_date`	`DATE`	NO	—	—	Fecha límite acordada para la devolución (`due_date ≥ loan_date`).
`return_date`	`DATE`	SÍ	—	`NULL`	Fecha real de reintegro físico (`NULL` mientras permanece prestado).
`status`	`VARCHAR(20)`	NO	—	`'ACTIVE'`	Estado del préstamo restringido a: `'ACTIVE'` , `'RETURNED'` , `'OVERDUE'` .
`active_book_id`	`INT`	SÍ	UK	`VIRTUAL`	Columna calculada: `IF(status = 'ACTIVE', book_id, NULL)` . Impide préstamos activos duplicados del mismo libro.
`notes`	`TEXT`	SÍ	—	`NULL`	Observaciones sobre el estado físico o entrega tardía del ejemplar.
`created_at`	`TIMESTAMP`	NO	—	`CURRENT_TIMESTAMP`	Marca temporal exacta de asentamiento de la transacción.

6. IMPLEMENTACIÓN DE REGLAS DE NEGOCIO Y RESTRICCIONES EN SQL

El diseño de persistencia traslada las reglas de negocio especificadas en la Fase 2 (IEEE 830) hacia restricciones declarativas (*Constraints*) ejecutadas directamente por el motor MySQL InnoDB, evitando que errores de capa de aplicación comprometan la consistencia de los datos.

6.1. Matriz de Trazabilidad de Reglas de Negocio a Constraints SQL

Tabla 7. Trazabilidad entre reglas de negocio (RN-01 a RN-05) y restricciones físicas en SQL.

Código Regla	Regla de Negocio	Mecanismo de Control Implementado en Base de Datos
RN-01	Unicidad de Código ISBN	Índice único declarativo `CONSTRAINT uk_books_isbn UNIQUE (isbn)` en la tabla `books`.
RN-02	Unicidad de Código de Autor	Índice único declarativo `CONSTRAINT uk_authors_code UNIQUE (author_code)` en la tabla `authors`.
RN-03	Unicidad de Documento de Identidad	Índice único declarativo `CONSTRAINT uk_users_document UNIQUE (document_number)` en la tabla `users`.
RN-04	Habilitación de Usuario Activo	Atributo booleano `users.is_active = TRUE` verificado antes de registrar transacciones en `loans`.
RN-05 (Parte A)	Prohibición de Préstamo Simultáneo	Columna virtual generada `active_book_id` con restricción `CONSTRAINT uk_loans_active_book UNIQUE (active_book_id)`.
RN-05 (Parte B)	Baja Lógica y Preservación de Historial	Política `ON DELETE RESTRICT` en `fk_loans_books` combinada con el borrado lógico `UPDATE books SET is_deleted = TRUE WHERE is_available = TRUE`.

6.2. Justificación Arquitectónica del Estado OVERDUE y Columna Virtual

1. Blindaje de préstamo único mediante columna generada (`active_book_id`): En el estándar SQL, los valores `NULL` no generan colisión en un índice `UNIQUE`. Al definir la columna virtual `active_book_id INT GENERATED ALWAYS AS (IF(status = 'ACTIVE', book_id, NULL)) VIRTUAL`, cuando un libro tiene un préstamo en curso (`status = 'ACTIVE'`), la columna adopta el valor numérico de `book_id`. Si se intenta insertar un segundo préstamo activo para el mismo ejemplar, el motor MySQL aborta inmediatamente la transacción por violación de clave única. En cambio, cuando el libro es devuelto (`status = 'RETURNED'`), el valor pasa a `NULL`, permitiendo que el libro sea prestado nuevamente en el futuro Tantas veces como sea necesario sin perder su historial.

2. Conservación del estado `OVERDUE` en el dominio de `loans.status`: La restricción `CHECK (status IN ('ACTIVE', 'RETURNED', 'OVERDUE'))` permite tanto el cálculo dinámico de mora en

consultas operativas (comparando `CURRENT_DATE > due_date`) como el etiquetado formal de préstamos vencidos en procesos de auditoría y penalizaciones futuras contempladas en la historia de usuario HU-10.

7. SCRIPT DDL/DML E INSTRUCCIONES DE DESPLIEGUE (MYSQL Y DOCKER)

El artefacto ejecutable oficial se encuentra versionado en la ruta `database/schema.sql` del repositorio e incluye la creación de la base de datos `digital_library`, las 5 tablas en 3FN, índices de optimización, carga de datos semilla (*Seed Data*) y cuatro bloques de consultas de validación funcional.

7.1. Extracto de Definición Estructural DDL (database/schema.sql)

```
-- Tabla Transaccional: loans (Préstamos y Devoluciones)
CREATE TABLE loans (
    id INT AUTO_INCREMENT,
    book_id INT NOT NULL,
    user_id INT NOT NULL,
    loan_date DATE NOT NULL,
    due_date DATE NOT NULL,
    return_date DATE NULL DEFAULT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
    active_book_id INT GENERATED ALWAYS AS (IF(status = 'ACTIVE', book_id, NULL)) VIRTUAL,
    notes TEXT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT pk_loans PRIMARY KEY (id),
    CONSTRAINT chk_loans_status CHECK (status IN ('ACTIVE', 'RETURNED', 'OVERDUE')),
    CONSTRAINT chk_loans_dates CHECK (due_date >= loan_date),
    CONSTRAINT uk_loans_active_book UNIQUE (active_book_id),
    CONSTRAINT fk_loans_books FOREIGN KEY (book_id)
        REFERENCES books (id) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_loans_users FOREIGN KEY (user_id)
        REFERENCES users (id) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

7.2. Procedimiento de Despliegue Local y en Contenedor Docker

Opción A — Despliegue automatizado con Docker Compose (Recomendado para Fase 5 y 6):

Atendiendo la recomendación técnica de la evaluación de Fase 4, el repositorio dispone del archivo `docker-compose.yml` en la raíz del proyecto, el cual levanta una instancia aislada de MySQL 8.0 e inyecta automáticamente el esquema `database/schema.sql` al inicializar el contenedor:

```
# 1. Levantar el contenedor MySQL 8.0 en segundo plano
docker compose up -d

# 2. Verificar la ejecución de las tablas y datos semilla
docker exec -it biblioteca_mysql mysql -u root -proot1234 digital_library -e "SHOW TABLES;"
```

Opción B — Ejecución directa en consola cliente MySQL o MySQL Workbench:

```
# Importación directa del esquema y semillas de prueba desde terminal
mysql -u root -p < database/schema.sql
```

8. CONCLUSIONES DE LA FASE

El desarrollo y verificación de la Fase 4 del **Sistema de Biblioteca Digital** permite establecer las siguientes conclusiones técnicas de ingeniería:

1. **Cumplimiento Estricto de la Tercera Forma Normal (3FN):** La estructuración del modelo relacional en cinco entidades cohesivas (`categories`, `authors`, `books`, `users` y `loans`) erradicó por completo la redundancia de datos maestros y las dependencias transitivas, garantizando que toda actualización sobre autores, categorías o usuarios mantenga consistencia inmediata en todo el sistema.
2. **Alineación con Estándares Internacionales de Codificación:** La adopción de la convención de nombres de tablas en plural e inglés junto con atributos en singular y formato *snake_case*, respaldada por la matriz canónica de equivalencia Español-Inglés, asegura una integración limpia con arquitecturas de persistencia en Java (JDBC / DAO / JPA) para la Fase 6 sin perder trazabilidad con los artefactos de análisis de las Fases 1 a 3.
3. **Resolución Arquitectónica entre Integridad Referencial y Baja Lógica:** La combinación de la política declarativa `ON DELETE RESTRICT` en las claves foráneas con la bandera booleana `is_deleted` en la entidad `books` resolvió satisfactoriamente el requerimiento de dar de baja ejemplares extraviados o deteriorados (HU-08) sin destruir el historial transaccional de préstamos requerido para auditoría.
4. **Blindaje Físico de Reglas de Negocio Críticas:** La implementación de restricciones avanzadas en el motor InnoDB —destacando la columna virtual generada `active_book_id` con restricción `UNIQUE` y las validaciones de dominio `CHECK` para rangos de años de publicación y fechas de entrega— garantiza que las reglas de negocio RN-01 a RN-05 se cumplan de forma ineludible a nivel de motor de base de datos.
5. **Preparación para Despliegue Reproducible:** La verificación de las suites de consultas DML sobre el script `database/schema.sql` y su empaquetado mediante contenedor Docker proporcionan al equipo de desarrollo una base sólida, estandarizada y libre de fricciones para abordar las fases de prototipado de interfaz (Fase 5) y codificación backend (Fase 6).